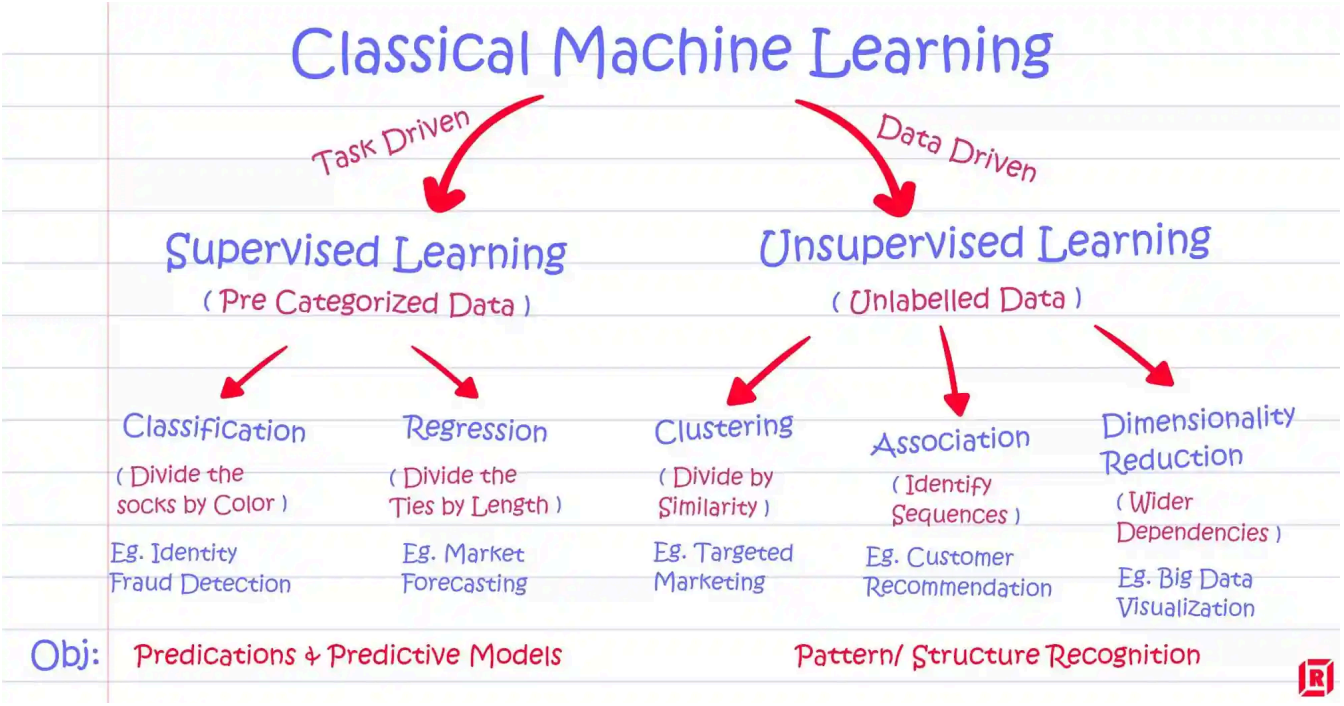
Session 02 - Intro to Machine Learning

Types of Machine Learning Problems

Machine Learning (ML) encompasses various problem types, primarily distinguished by the nature of the data (labeled or unlabeled) and the type of output (discrete or continuous).

	Supervised Learning	Unsupervised Learning
Discrete	Classification or Categorization	Clustering
Continuous	Regression	Dimensionality Reduction

> **Regression ≠ Classification**
>
> Do not confuse **regression** with **classification** just because both are “supervised learning.”
>
> - Classification → predict **categories** (like cats, dogs, birds)
> - Regression → predict **continuous values** (like angles, prices, or durations)



> # Supervised vs. Unsupervised Learning

Supervised vs. Unsupervised Learning

- **Supervised Learning** training a model on a labeled dataset. Each training example is paired with an output label. The model learns to predict the output from the input data.
- **Unsupervised Learning** deals with unlabeled data. The model tries to learn the underlying structure from the input data without explicit instructions on what to predict.

Discrete vs. Continuous Data

- **Discrete Data**: Represents distinct and separate values. For example, classifying animals as 'domestic' or 'wild' is a discrete task.

- **Continuous Data**: Represents measurements and can take any value within a range. For instance, predicting the angle of a finger on a touchscreen (0° to 180°) is a continuous task.

Supervised Learning: Training on Labeled Data

Let's say we have a dataset of animal emojis, each labeled: `"cat"`, `"horse"`, `"dog"`, `"gorilla"`, etc. We train a model on these **labeled examples**. During training, the model learns **which weights and biases** produce the correct decisions.

Once trained, the model can make predictions on **unseen data**; e.g., it receives a new emoji and classifies it as a "dog" based on the patterns it learned.

Unsupervised Learning: Clustering Without Labels

Now we remove the labels and simply look at the emojis. We want the model to **find patterns or groupings** without telling it what each emoji represents.

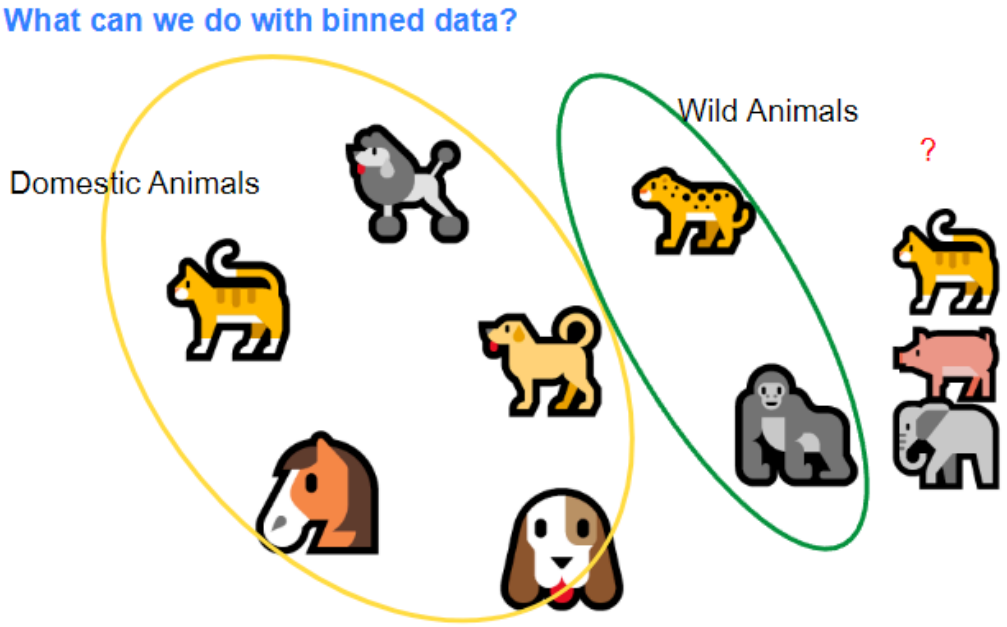
One clustering idea could be:

- Group emojis by **physical features**, such as "has legs" vs. "no legs" (e.g., 🐕 vs. 🐍).
- Another idea: Cluster by **head vs. full-body** images.

What can we learn just from looking at the data?
We can observe visible traits like:

- Full animal vs. just a head.
- Has legs or no legs.
- Possibly infer if it's a domestic or wild animal.

These visual features can be the **basis for clustering**, even without any labels.



What about Binned Data?

→ Let's say we **manually group** (or "bin" = 🚮) animals into categories like `"domestic"` and `"wild"`. This adds structure to the dataset.

What can we do with binned data?

- Analyze how many animals fall into each category.
- Compare domestic vs. wild animals statistically.
- Train a **supervised model** if we use the bin labels as targets.

What happens if we add more unlabelled animals to the binned dataset?
The new animals won't have a category yet. A supervised model **cannot use them for training**, but an unsupervised algorithm might try to:

- **Infer new groupings**, potentially suggesting a third cluster.
- **Alter existing clusters** if it sees the new data as similar to existing groups.

This is how unsupervised learning adapts but it's also **unstable**: each run may result in different groupings.

> **We can combine learning types**
>
> One strategy is to first use **unsupervised learning** to cluster unknown data, then **manually label** those clusters and switch to **supervised learning** for a more stable model.

> **What happens if we mix labeled and unlabeled data?**
>
> We can not use supervised learning directly with unlabeled data, because supervised learning needs labels to learn from.
> But we can use a **new approach** called **semi-supervised learning**; it combines both labeled and unlabeled data. Here's how:
>
> - The model learns from the small labeled set.
> - Then it tries to **guess labels** for the unlabeled data based on what it learned.
> - These guesses can then be used to improve the model further.

Who’s the Odd One Out?

This leads to a common goal in unsupervised learning:
→ **Find anomalies or the “odd one out”** in a dataset.

The model does this by:

- Discovering patterns or clusters.
- Identifying data points that **don’t fit** any pattern well.

Why is this difficult for humans?
Because the clusters created:

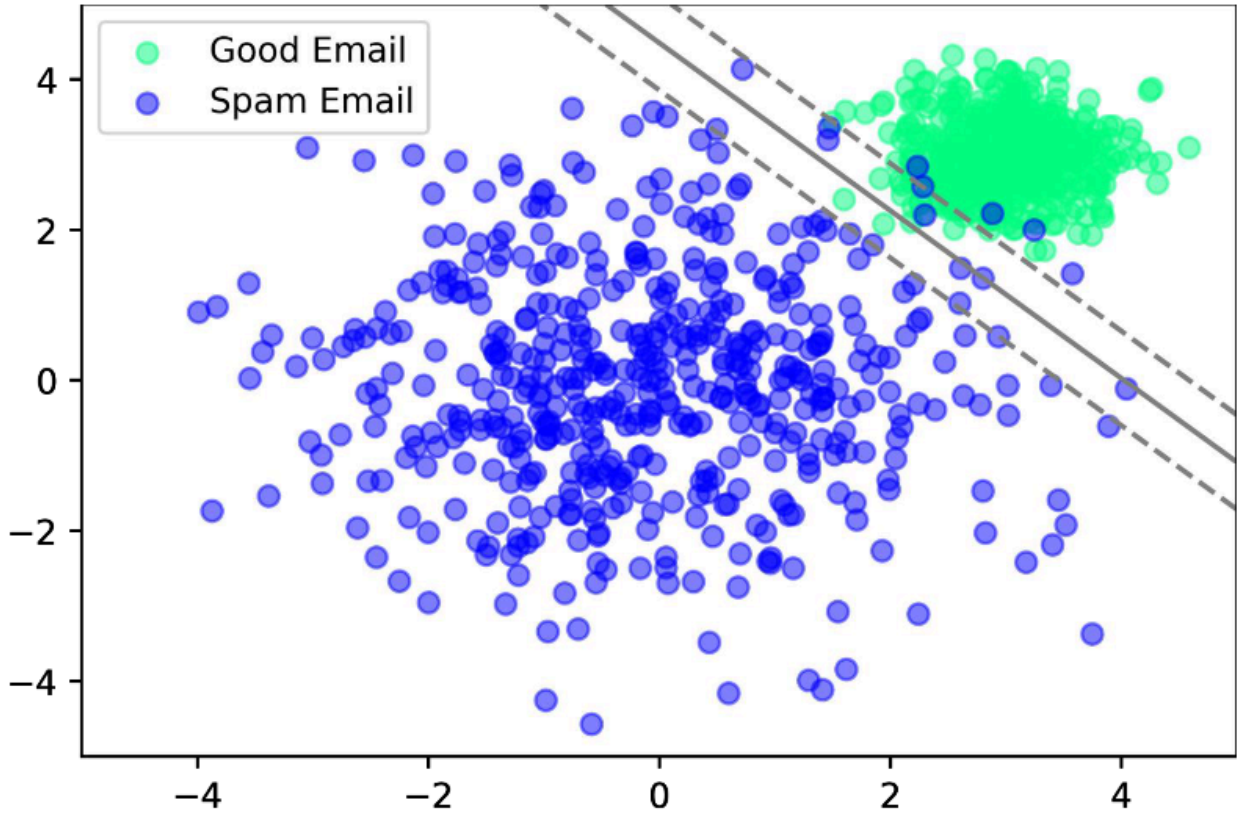
- Don’t have **meaningful names**.
- May change **every time** the algorithm runs.
- Require humans to manually **interpret or relabel** the clusters.
- If we ask the model to cluster in ten bins, we get ten bins, even if this does not make real sense.

That’s why this approach is **less suitable** for “human-in-the-loop” workflows, where we need **consistency and interpretability**.

Example: Email Classification with SVM

As said before **supervised learning** algorithms are trained using labeled datasets. The model makes predictions based on input data and adjusts its parameters to minimize the difference between its predictions and the actual labels. This is useful for labeling data like mails to dtect spam.

We can plot these data points in a coordinate system where each point represents an email, characterized by features like word frequency, sender reputation, etc. Emails are labeled as 'spam' (blue) or 'not spam' (green). A **Support Vector Machine (SVM)** can be used to find the optimal boundary (black line) that separates the two classes.

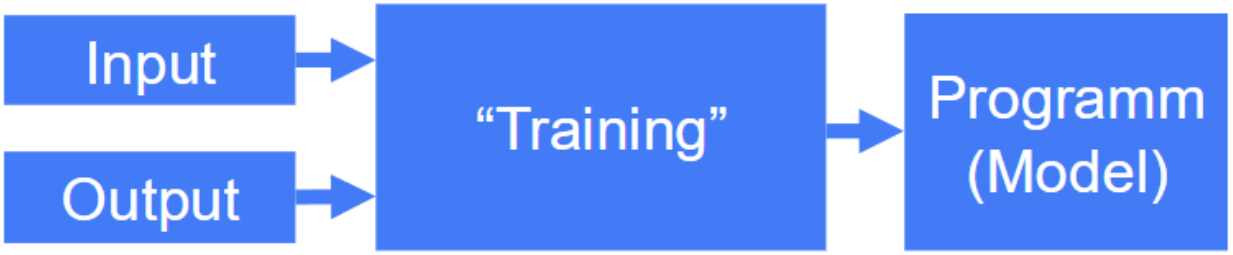


- **Support Vectors**: The **data points closest to the decision boundary**. These points lie on the edge of the margin and are crucial because they **directly influence the position and orientation** of the separating line (hyperplane). If we removed them, the boundary would shift.
- **Margin**: The **distance between the decision boundary and the nearest support vectors** from each class. SVM tries to **maximize this margin**, creating the widest possible separation between the classes to improve generalization.
    - Decision Boundary: "Emails on this side are spam; emails on that side are not spam. Emails exactly on the line are uncertain the probability of spam/not spam is 50/50"

> **Support Vector Machine - Wikipedia**
>
> "A support vector machine constructs a hyperplane or set of hyperplanes in a high or infinite-dimensional space, which can be used for classification, regression, or other tasks like outliers detection."

"Support Vector Machines (SVMs) require input data and their associated labels (expected outputs) during training."
Explanation:

- "Input" refers to the features (e.g., word frequency, sender reputation).
- "Expected output" refers to the correct label (e.g., spam or not spam).
- This setup describes **supervised learning**, which SVMs use.



Why are support vectors so important and what would happen to the decision boundary if removed?

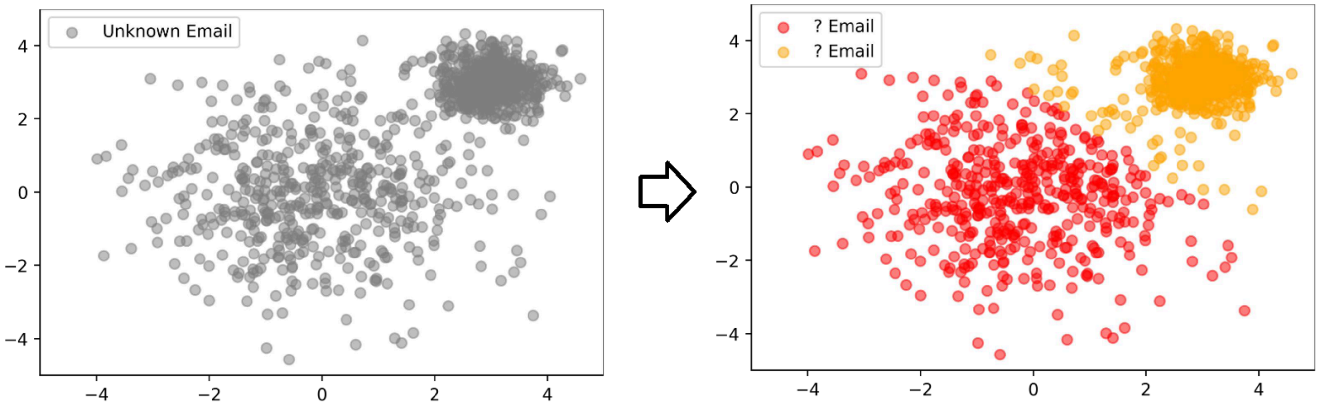
Support vectors are the **closest data points** to the decision boundary (the separating line between classes in SVM).
They are important because:

- The position of the boundary **depends directly** on them.
- If a support vector is removed or moved, the boundary would **change**.

Removing **non-support vectors** means that the boundary would likely **stay the same**. But removeing a support vector means that, the whole model might shift its decision line, possibly leading to **wrong predictions**.

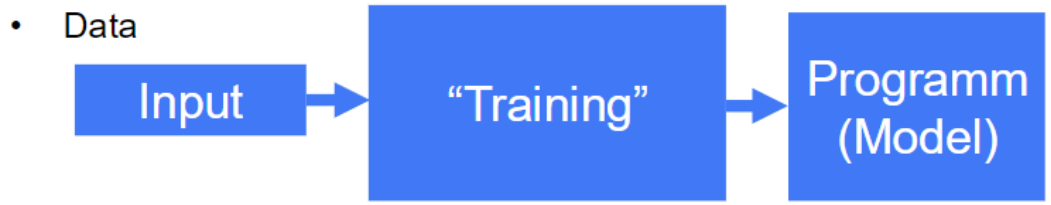
Unsupervised Learning

In unsupervised learning, we only provide input data without labels, and the model tries to find hidden patterns or groupings within the data."



This contrasts with supervised learning because:

- There's **no expected output**.
- The model must figure out structure on its own, e.g. clustering similar items together.



Not needed:

- Labels (Classes)
- Continues values

Clustering

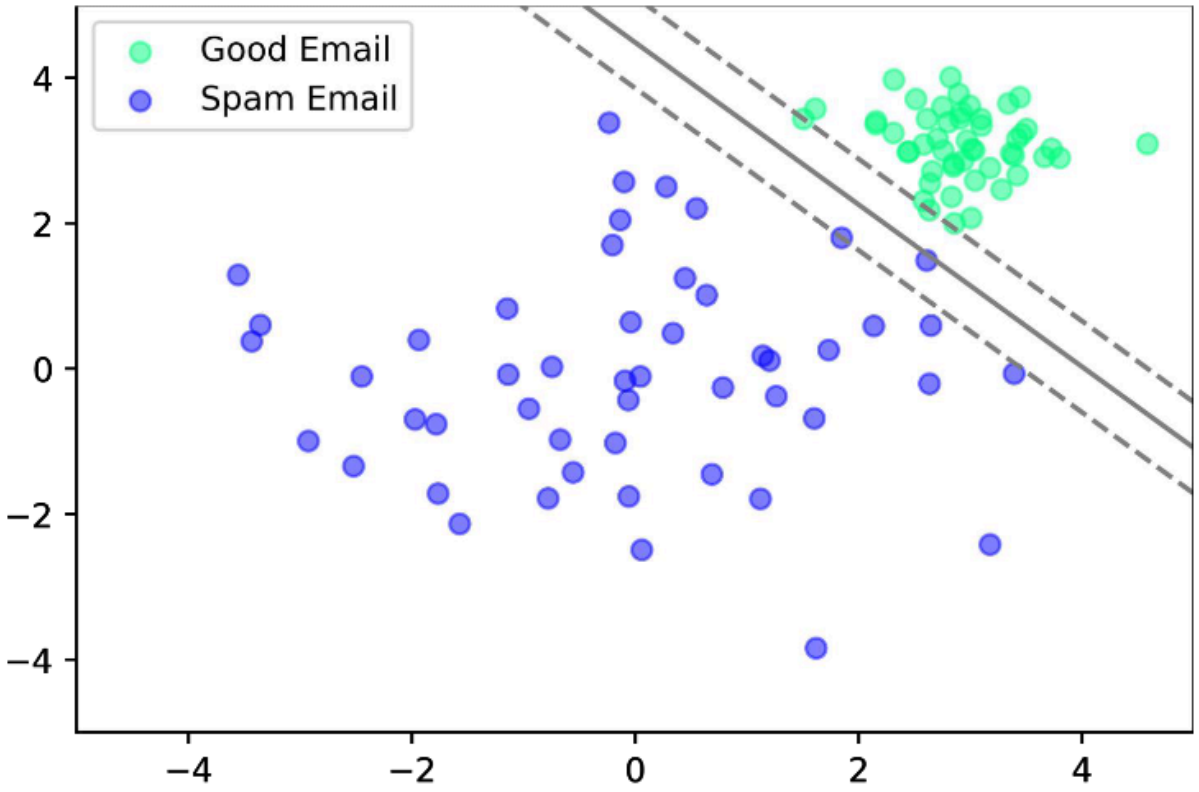
Clustering involves grouping data points based on similarity. For example, grouping animal emojis based on features like the presence of legs or habitat.

- **Challenge**: The model assigns arbitrary labels to clusters, which may not be meaningful without human interpretation.

> **Human-in-the-loop Approach**
>
> In unsupervised learning, incorporating human feedback can help assign meaningful labels to clusters, enhancing the model's utility.

Classification

Classification is a supervised learning task where the model predicts discrete labels.



One-Hot Encoding

One-hot encoding is a method to convert **categorical labels** (like spam categories) into a numerical format that machine learning models can understand.

In our **email classification example**, we label each email as either **spam** or **not spam**. Since these are categories (not numbers), we use one-hot encoding to represent them numerically:

- **Spam** → [1, 0]
- **Not Spam** → [0, 1]

This allows us to **plot emails in a coordinate system**, where each email is a point defined by:

- Its **features** (e.g. word frequency, sender reputation) → used as coordinates in feature space
- Its **label** (spam or not) → represented using one-hot encoding

So, while the **position** of an email in the feature space is based on numeric values like "how often the word 'offer' appears", the **class** it belongs to (spam or not) is stored using a binary vector.

> **One-Hot Encoding in NLP - GeeksforGeeks**
>
> "One-hot encoding is the process of turning categorical factors into a numerical structure that machine learning algorithms can readily process."

Input and Output Vectors

In the email classification example:

- **Input Vector (Features)**: Represents the characteristics of an email, e.g., word frequencies, sender reputation.
- **Output Vector (Label)**: Represents the class label, e.g., [1, 0] for 'spam', [0, 1] for 'not spam'.

Non-Linear Classification and Overfitting

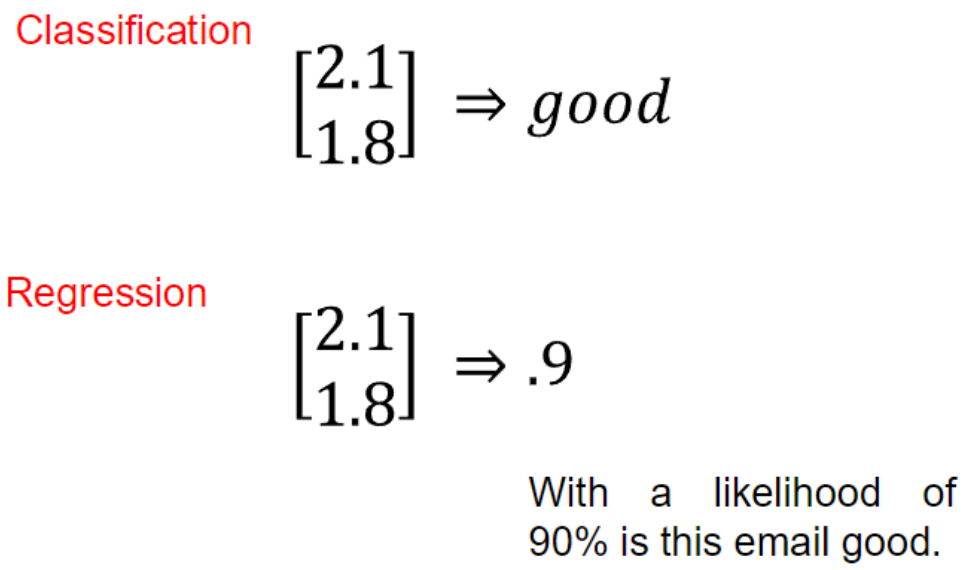
- **Non-Linear Classification**: When data is not linearly separable, SVMs can use kernel functions to project data into higher-dimensional spaces where a linear separator is feasible.
- **Overfitting**: Occurs when the model learns noise in the training data, leading to poor generalization on unseen data. It's evident when misclassified points (e.g., blue dots in the green cluster) are present.

> **Avoiding Overfitting**
>
> Techniques like cross-validation, regularization, and pruning can help prevent overfitting by ensuring the model generalizes well to new data.

> **Overfitting looks like “perfection”**
>
> A model that classifies **100% of training data "correctly"** might be **overfitting**. This means it might be *memorizing noise instead of learning general patterns*.

Regression

Regression is a supervised learning task where the model predicts continuous outcomes.



> "There are not necessalry good or bad mails, but there is a likelyhood if a mail is good or bad."

Support Vector Regression (SVR)

SVR is an extension of SVM for regression problems. It tries to fit the best line within a threshold (epsilon) that captures most data points.

- **Linear SVR**: Fits a straight line to the data, minimizing the error within the epsilon margin.
- **Non-Linear SVR**: Uses kernel functions to capture complex relationships.

> **Support Vector Machine - Wikipedia**

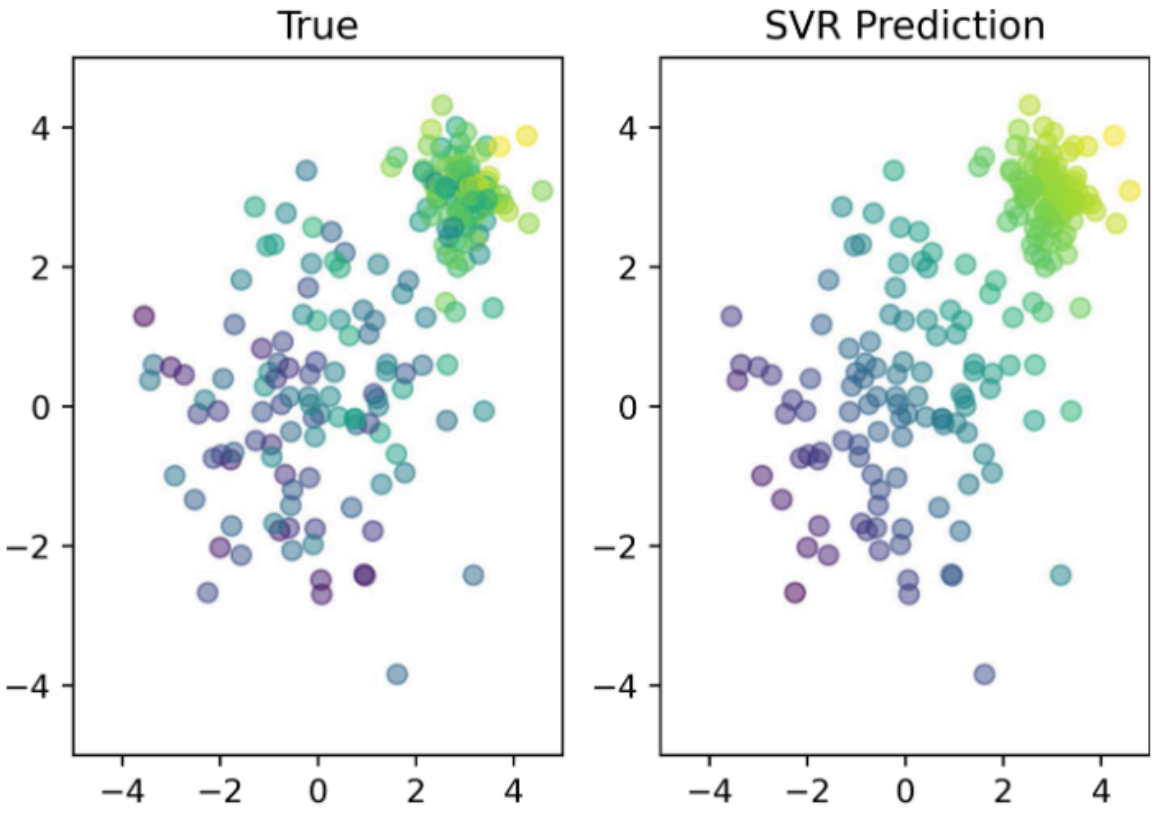
"Support vector machines can efficiently perform non-linear classification using the kernel trick, implicitly mapping their inputs into high-dimensional feature spaces."

Support Vector Regression

Linear SVR fits a (linear / non linear) line to the data while ignoring errors smaller than a threshold ε, and only penalizing predictions that are **outside** this margin. (*see image 2 below with non linar RBF-SVR prediction*)

SVR in 2D Space

In a 2D space, SVR predicts a continuous value for each data point. The gradient between points indicates the direction and rate of change.



Understanding SVR Output

The color gradient in SVR plots represents the predicted values, with different shades indicating varying levels of the target variable.

Radial Basis Function (RBF) Kernel

The **RBF kernel** (also called the **Gaussian kernel**) is a popular choice for non-linear SVM and SVR models. It maps input features into higher-dimensional spaces, allowing the model to capture complex patterns. It measures similarity between two data points `x` and `x'`:

SVR Model Complexity

The power of an SVR (Support Vector Regressor) depends on the **kernel** used. A **Linear SVR** fits only a straight line; so its "expressive power" is limited - just spam or not spam. In contrast, an **SVR with a non-linear kernel** (like RBF or polynomial) can fit **curved patterns**, even with multiple bends or sections (e.g. 6 segments in a polynomial curve). This allows it to model complex relationships **without necessarily overfitting**.

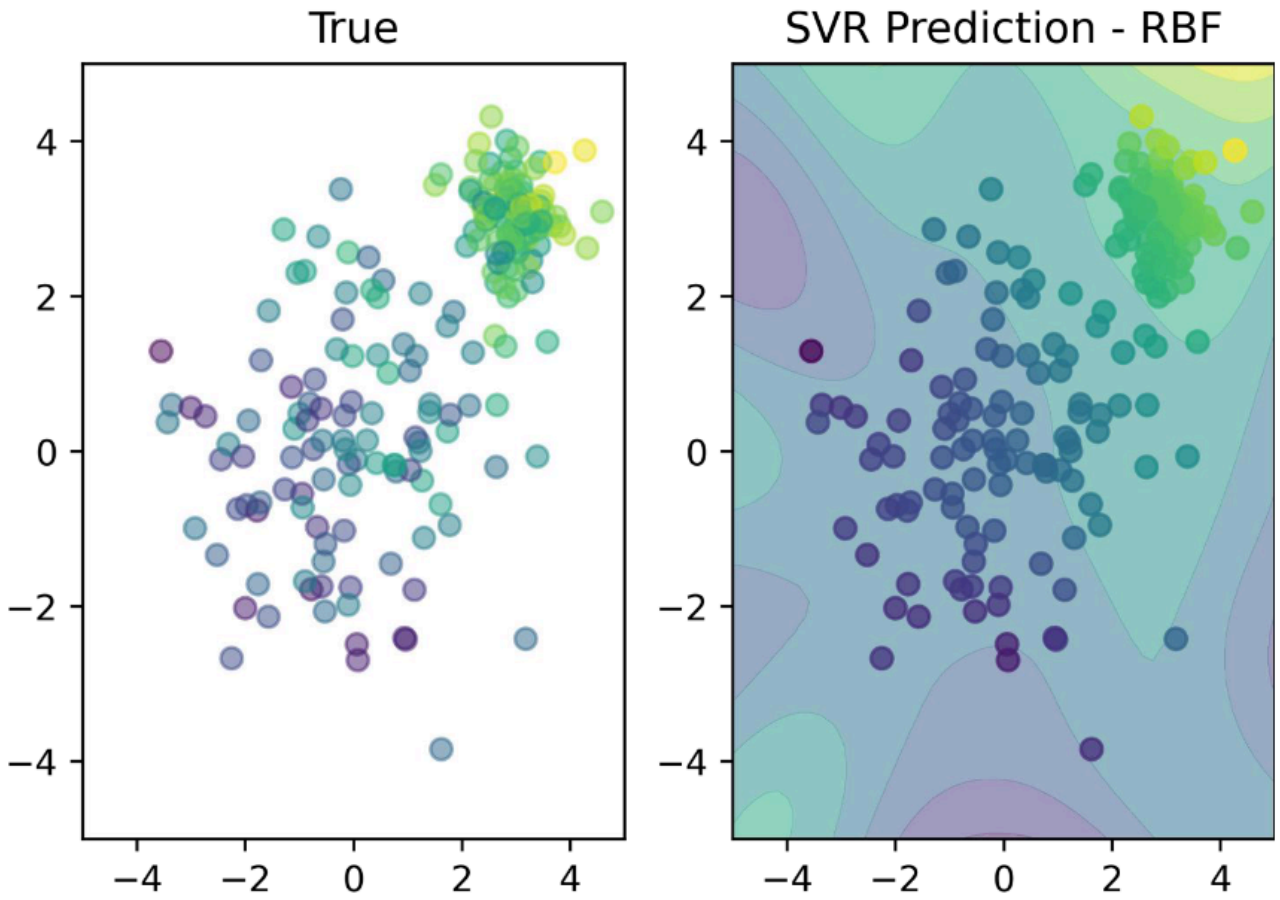
Explaining the Image:
The coordinate system shows **data points (here emails)**,

- The **color gradient from blue to green** represent the **SVR's predicted value or score** for each point. E.g., spam probability
- The **shaded regions** around the points (blue-to-green) show **model confidence or decision boundaries**: areas where similar values are predicted. The color of the region corresponds to the predicted values of nearby dots.

Takeaway:
The use of color gradients and complex regions in the image suggests that the SVR is capturing **subtle patterns** in the data, which is only possible with a **non-linear SVR** (like RBF or polynomial). This flexibility is what gives SVR its **real predictive power**.

Where:

- `γ` (gamma) defines the **influence range** of a single training example.
- Larger γ means **narrower influence** (more complex models).
- Smaller γ means **broader influence** (smoother models).
- **Definition**:
  `K(x, x') = exp( −γ * ||x − x'||² )`
  where γ is a parameter that defines the influence of a single training example.



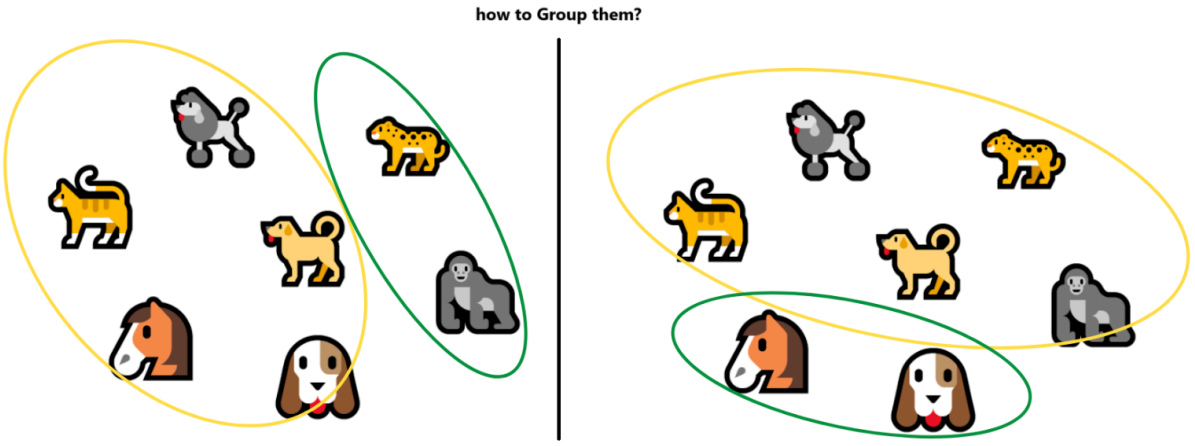
> Radial Basis Function Kernel - Wikipedia
>
> "The RBF kernel on two samples x and x', represented as feature vectors in some input space, is defined as $K(x, x') = \exp(-\gamma||x - x'||^2)$."

> **Choosing γ in RBF Kernel**
>
> A small γ value implies a larger similarity radius, leading to smoother decision boundaries. A large γ value focuses more on the exact match, potentially leading to overfitting.

Unsupervised Learning

Unsupervised learning deals with unlabeled data.
The primary goal is to find hidden patterns, structures, or relationships within the input data without explicit guidance or pre-defined output labels. This contrasts with supervised learning, where models are trained on labeled datasets with known outcomes.

- **Clustering**: This technique is used for finding inherent groupings or structures within unlabeled data, such as identifying different categories of emails without prior knowledge of what those categories are.
- **Dimensionality Reduction**: This involves transforming high-dimensional data into a lower-dimensional space, effectively reducing the data to its more essential features. This not only simplifies the data but also helps in visualizing complex datasets and mitigating the "curse of dimensionality".



> Unlike supervised learning, unsupervised learning does not require labels or continuous values as part of the input.

Learning Strategies

Machine learning problems can be categorized based on whether they involve discrete or continuous data and whether they fall under supervised or unsupervised learning.

	Supervised Learning	**Unsupervised Learning**
Discrete	Classification or Categorization	Clustering

	Supervised Learning	Unsupervised Learning
Continuous	Regression	Dimensionality reduction

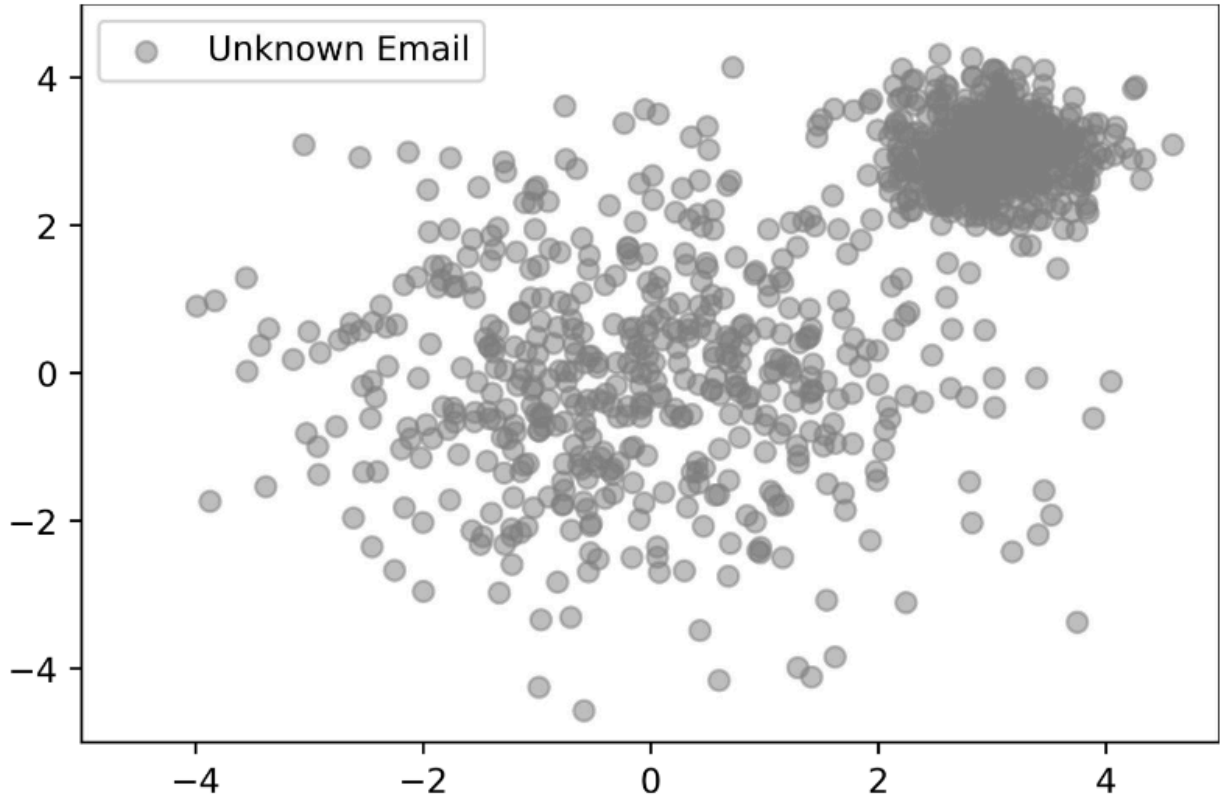
Unsupervised Learning Methods

Several methods are employed in unsupervised learning to uncover patterns and structures in data. These include:

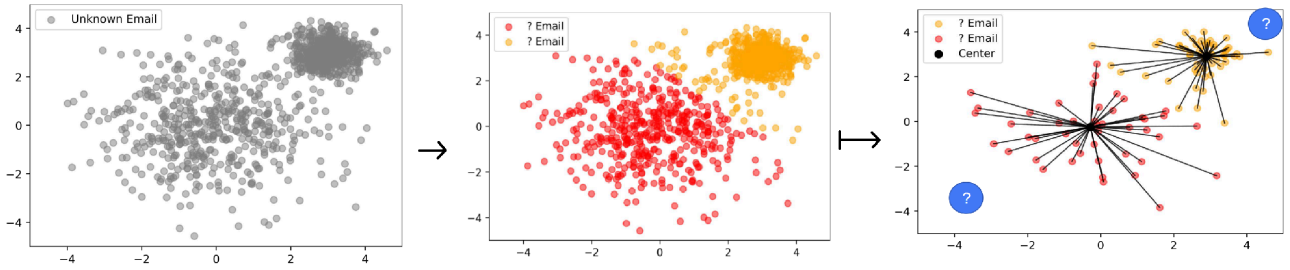
- Hierarchical clustering
- K-means clustering
- Principal Component Analysis (PCA)
- Singular Value Decomposition
- Independent Component Analysis

Clustering

Clustering is a core unsupervised learning technique where the goal is to group a set of objects in such a way that objects in the same group (cluster) are more similar to each other than to those in other groups. This method is particularly useful when dealing with unknown data, such as a collection of emails where the content or sender is initially unknown.



The process of clustering can transform seemingly unstructured "Unknown Email" data into groups. For instance, K-means clustering is an algorithm that uncovers "structure" in unlabeled data by partitioning it into a predefined number of clusters.



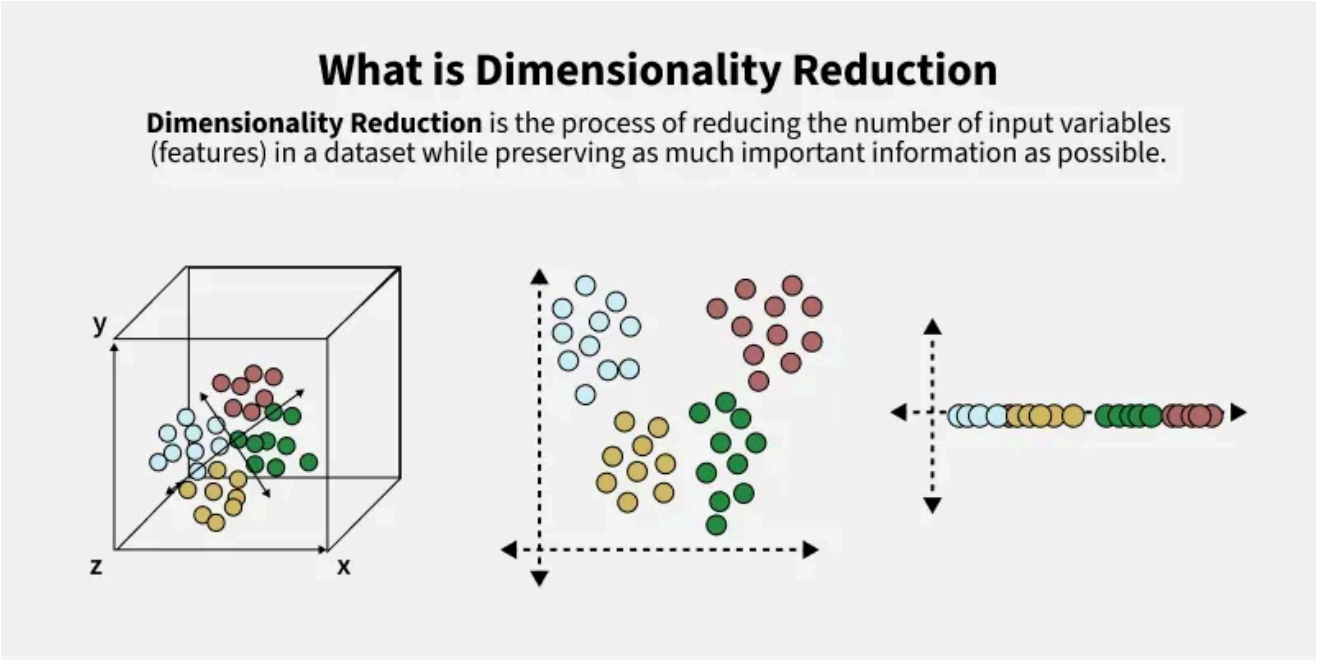
What does the center in the graph that is clustering the different emails represent?

The 'Center' in a K-means clustering graph (often called a centroid) represents the mean position of all data points belonging to that specific cluster. The distance from a data point to a cluster center indicates how similar that data point is to the other members of that cluster. A shorter distance implies greater similarity and a stronger association with that cluster. When a new data point is introduced, it will be assigned to the cluster whose center is closest to it.

The objective in K-means clustering is to minimize the sum of squared distances between data points and their assigned cluster centroids. This **iterative process** aims to find the optimal center for each cluster. Initially, two points might define a preliminary center, and as more points are added, the cluster center dynamically adjusts to minimize the overall sum of distances within the cluster.

Dimensionality Reduction

Dimensionality reduction is an unsupervised learning technique that involves transforming high-dimensional data into a lower-dimensional space. This process is crucial for simplifying data, reducing noise, and making it easier to visualize and analyze.



> Dimensionality Reduction

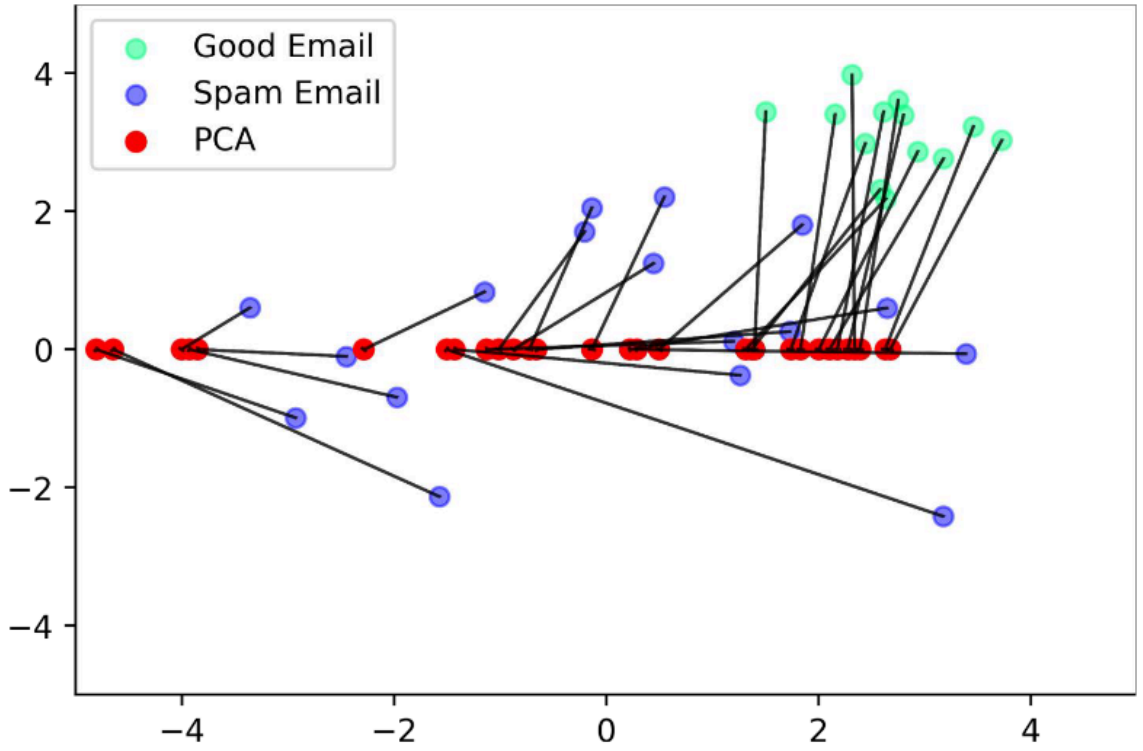
One of the prominent methods for dimensionality reduction is **Principal Component Analysis (PCA)**. PCA aims to reduce the number of features (dimensions) in a dataset while retaining as much variance as possible.

This means that if we have n features $(x_1, \ldots, xn)$ where n is large, PCA can reduce it to m features $(x_1, \ldots, x_m)$ where m is significantly smaller than n. $(n > m)$.

Dimensionality Reduction:

Dimensionality reduction is like compressing a large, detailed map into a smaller, simpler one that still shows the most important landmarks. It's about finding the essential information in complex data and discarding the rest without losing too much meaning.

For example, in an email dataset, PCA can reduce numerous features (like individual word frequencies, sender information, etc.) into a few principal components that still effectively represent the underlying information, helping to distinguish between different types of emails.



> PCA can simplify the representation of data while preserving critical information, which can be useful for tasks like identifying 'Good Email' versus 'Spam Email' by projecting them onto a lower-dimensional space.

PCA is unsupervised

PCA is an **unsupervised method**, which means it doesn't know any correct labels or categories. It just tries to keep the most important patterns in the data but those patterns might not match the intended categories.
